

Universe Reinsurance Platform

Work Environment Deployment Specification

Server, Database, Network, Security, Monitoring, and Go-Live Checklist

Version 2.0 — July 2026 — Confidential

Supersedes Version 1.0 (April 2026)

Document Control

Field	Detail
Document name	Universe Reinsurance Platform — Work Environment Deployment Specification
Version	2.0 (supersedes 1.0, April 2026)
Date	July 2026
Classification	Confidential — For Internal IT Use Only
Scope	Server, database, network, security, monitoring, and go-live readiness
Source repository	<code>https://github.com/Darchville-Analytics/modelling_tool</code>
Living companion doc	<code>DEPLOYMENT.md</code> in the repository root — always confirm against the latest code before relying on this snapshot

What Changed Since Version 1.0

Area	v1.0 (April 2026)	v2.0 (current)
Repository	<code>github.com/iwadaya/claude_universe</code>	<code>github.com/Darchville-Analytics/modelling_tool</code> (org-owned, private)
Application port	3000	4000 (default in <code>.env.example</code>)
Auth secret variable	<code>JWT_SECRET</code>	<code>AUTH_JWT_SECRET</code> — production refuses to boot without a strong (≥ 32 char) value
Migrations	<code>npm run migrate</code>	<code>npm run migrate:status</code> / <code>npm run migrate:up</code> (<code>--prefix server</code>); run as an explicit pre-deploy step, <code>RUN_MIGRATIONS_ON_BOOT=false</code> in production
PM2	Manual <code>pm2 start</code> flags, fixed 4 instances	Bundled <code>ecosystem.config.cjs</code> (auto instance count, derived <code>DB_POOL_MAX</code> , graceful reload)
Nginx pattern	Nginx serves <code>client/dist</code> ; proxies <code>/api</code> only	Express serves the built SPA itself; Nginx proxies everything to <code>127.0.0.1:4000</code> ; <code>client_max_body_size 50M</code>
Authentication	Demo-era auth	Real scrypt credential check + signed bearer tokens; enforcing CSP; 12-char password policy; optional enterprise SSO/MFA
New production requirements	—	<code>CORS_ORIGIN</code> , durable upload storage (<code>CLOUDINARY_URL</code> or <code>ALLOW_LOCAL_UPLOADS=true</code>), <code>REDIS_URL</code> for cluster mode, optional ClamAV scanning
Backups	Hand-written <code>pg_dump</code> cron	Checked-in, restore-verified scripts: <code>scripts/backup-db.sh</code> , <code>scripts/verify-restore.sh</code> , <code>scripts/universe-backup.cron</code> , <code>scripts/backup-uploads.sh</code>
Alternative runtimes	PM2 only	PM2 + Nginx (recommended), Docker Compose, or systemd — see <code>DEPLOYMENT.md §6</code>

1. Deployment Overview

This specification defines the requirements for deploying the Universe Reinsurance Platform (repository name: `modelling_tool`) in a corporate work environment. It is intended for internal IT administrators responsible for provisioning, configuring, securing, monitoring, and releasing the platform into production. Follow the sections in sequence to achieve a clean and repeatable deployment.

1.1 Platform Stack

Layer	Technology	Version	Purpose
Frontend	React + Vite	React 19 / Vite 6.x	Browser-based SPA compiled to static assets, served by the API server from <code>client/dist</code>
Backend	Node.js + Express	Node.js 20.18.0 (pinned in <code>.nvmrc</code> / <code>.node-version</code>) / Express 5	REST API server, business logic, and static SPA hosting
Database	PostgreSQL	14+ (16 or 17 recommended)	Persistent application data storage
Process manager	PM2	Latest stable	Cluster mode, on-boot start, graceful reload (config bundled in repo)
Reverse proxy	Nginx	1.24+	HTTPS termination and proxying to the app on port 4000
Rate-limit store	Redis / Valkey	Any current	Required for multi-instance / PM2 cluster deployments (shared rate limits)
Source control	Git + GitHub	Git 2.34+	Code repository and deployment source

The Express server serves both the API under `/api/*` and the built SPA. **No separate frontend host is required.**

2. Server Requirements

2.1 Sizing (approx. 30 concurrent users)

Resource	Minimum	Comfortable / recommended
CPU	2 vCPU	4 vCPU (PM2 cluster uses up to 4 workers by default)
RAM	4 GB (Postgres on a separate host) / 8 GB (co-located Postgres)	8–16 GB
Disk	20 GB SSD	50 GB SSD; separate volume for PostgreSQL data in production
Backup storage	Off-host target for nightly dumps	Sized for ≥30 daily dumps + uploads archive
Network	1 Gbps NIC, static IP	Dual NICs where available

Note: these figures are a starting point; revise after running the k6 capacity tests in `load-test/` (see `docs/scaling.md`).

2.2 Operating System Baseline

Item	Requirement
Operating system	Ubuntu Server 22.04 LTS or 24.04 LTS, 64-bit
Filesystem	ext4 or XFS for data volumes
Swap	4–8 GB swap partition
Timezone	Set to the local timezone using <code>timedatectl set-timezone</code>
Locale	<code>en_US.UTF-8</code>
Automatic updates	Enable <code>unattended-upgrades</code> for security patches only

3. Software Requirements

3.1 Required Software Stack

Package	Requirement / installation notes
Node.js	20.18.0 — pinned in <code>.nvmrc</code> and <code>.node-version</code> ; use exactly what is there. Install via NodeSource (<code>curl -fsSL https://deb.nodesource.com/setup_20.x sudo -E bash -</code>) or <code>nvm</code> .
npm	npm 10.x, included with Node.js 20
PM2	<code>sudo npm install -g pm2</code> , then <code>pm2 startup</code>
PostgreSQL	14, 15, 16, or 17. <code>sudo apt install postgresql-16 postgresql-contrib</code>
Git	2.34+. <code>sudo apt install git</code>
Nginx	<code>sudo apt install nginx</code> for HTTPS and reverse proxying
Certbot	Only when using Let's Encrypt: <code>sudo apt install certbot python3-certbot-nginx</code>
Redis / Valkey	Only for PM2 cluster mode or multi-instance deployments: <code>sudo apt install redis-server</code>
ClamAV	Recommended for production upload scanning: <code>sudo apt install clamav-daemon</code>

3.2 Installation Sequence

1. Update the operating system: `sudo apt update && sudo apt upgrade -y`
2. Install Node.js 20.18.0 (NodeSource or `nvm`)
3. Install PostgreSQL and create the database user (§4)
4. Install Git and configure GitHub access (§6.1) — **password authentication no longer works; a token or SSH deploy key is required**
5. Clone the repository (§6.2)
6. Install PM2 globally; install Nginx; install Redis if clustering
7. Create and secure the `.env` file from `.env.example` (§5)
8. Run `npm run install:all`, then `npm run build`
9. Run database migrations: `npm run migrate:up --prefix server`
10. Start the application: `pm2 start ecosystem.config.cjs`
11. Configure the Nginx reverse proxy (§7)
12. Install and validate the SSL certificate
13. Schedule backups (§4.3) and verify health endpoints (§10)

4. Database Setup

4.1 PostgreSQL Installation and Initial Configuration

```
# Install PostgreSQL
sudo apt install -y postgresql-16 postgresql-contrib
sudo systemctl enable --now postgresql

# Create the database user and database
sudo -u postgres psql <<SQL
CREATE USER universe WITH PASSWORD 'CHANGE_ME_STRONG_PASSWORD';
CREATE DATABASE universe OWNER universe;
GRANT ALL PRIVILEGES ON DATABASE universe TO universe;
SQL
```

The resulting connection string is:

```
DATABASE_URL=postgresql://universe:CHANGE_ME_STRONG_PASSWORD@localhost:5432/universe
```

URL-encoding: if the password contains URL-special characters (@ : / # %), URL-encode them — e.g. Sahara@66 becomes Sahara%4066 .

For a separate database host, install `postgresql-client-16` on the app server, allow the app server's IP in `pg_hba.conf` on the DB host, and append `?sslmode=require` to `DATABASE_URL` .

4.2 Connection Sizing

Postgres `max_connections` defaults to 100. The app's per-process pool default is 50 (`DB_POOL_MAX`). Leave at least 30 connections free for migrations, ad-hoc queries, and monitoring. In PM2 cluster mode each worker has its own pool — `ecosystem.config.cjs` automatically derives a safe per-worker `DB_POOL_MAX` (total ≤ 80), but verify `workers × DB_POOL_MAX < max_connections` if you override anything.

For a 16 GB server, baseline tuning in `/etc/postgresql/16/main/postgresql.conf` : `shared_buffers = 4GB` , `effective_cache_size = 12GB` , `maintenance_work_mem = 1GB` , `checkpoint_completion_target = 0.9` , `wal_buffers = 64MB` , `random_page_cost = 1.1` , `work_mem = 40MB` . Scale these down proportionally on smaller hosts.

4.3 Backup Schedule (use the checked-in scripts)

The repository ships restore-verified backup tooling — do not hand-write `pg_dump` cron lines.

Script	What it does
<code>scripts/backup-db.sh</code>	Compressed <code>pg_dump -Fc</code> of <code>DATABASE_URL</code> into <code>BACKUP_DIR</code> (default <code>/var/backups/universe</code>); prunes dumps older than <code>BACKUP_RETENTION_DAYS</code> (default 30)
<code>scripts/verify-restore.sh</code>	Restores the newest dump into a throwaway scratch DB, sanity-checks it, and exits non-zero if the dump won't restore — catches silently corrupt backups
<code>scripts/backup-uploads.sh</code>	Backs up the uploads directory (slips and documents) alongside the database
<code>scripts/universe-backup.cron</code>	Host-cron schedule: nightly dump at 02:00 + weekly restore rehearsal (Sun 03:30)

```
# Install the schedule (set DATABASE_URL in the cron environment first)
crontab scripts/universe-backup.cron      # or copy into /etc/cron.d/

# One-off manual verification
DATABASE_URL=postgresql://universe:PASSWORD@localhost:5432/universe \
BACKUP_DIR=/var/backups/universe scripts/backup-db.sh
DATABASE_URL=postgresql://universe:PASSWORD@localhost:5432/universe \
BACKUP_DIR=/var/backups/universe scripts/verify-restore.sh  # exit 0 = restorable
```

Copy dumps to off-host storage daily, retain ≥ 30 daily dumps, and document a monthly restore test. A backup that has not been restored successfully is not a verified backup.

5. Environment Configuration

Copy the template and secure it. The `.env` file contains secrets, must be created on the server, and must never be committed to Git (it is in `.gitignore`).

```
cd /opt/universe
cp .env.example .env
nano .env          # fill in real values
chmod 600 .env
```

5.1 Key Environment Variables

Variable	Required	Notes
NODE_ENV	Yes	Set to <code>production</code>
PORT	Yes	App listens here. Default 4000 (was 3000 in v1.0)
DATABASE_URL	Yes	Postgres connection string (§4.1)
AUTH_JWT_SECRET	Yes (production)	Signs login bearer tokens. Must be ≥32 chars. The server refuses to start in production without a strong value. Generate: <code>node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"</code>
CORS_ORIGIN	Yes	The public URL, e.g. <code>https://universe.internal.company.com</code> . Never <code>*</code> in production
SESSION_TTL_HOURS	No	Token lifetime; default 8 (a working day)
RUN_MIGRATIONS_ON_BOOT	No	Set <code>false</code> in production; run migrations as an explicit pre-deploy step (§6.4)
CLOUDINARY_URL	Yes (production)	Durable private object storage for uploads. Without it, production uploads fail <code>503 STORAGE_NOT_DURABLE</code> . On a single box with a persistent volume you may instead set <code>ALLOW_LOCAL_UPLOADS=true</code>
CLAMAV_ENABLED	Recommended	<code>true</code> streams every upload to clamd before storage, fail-closed (infected → 422; scanner down → 503). Also <code>CLAMAV_HOST / PORT / TIMEOUT_MS</code>
REDIS_URL	Yes (cluster)	Shared rate-limit store. Required for PM2 cluster mode or multiple replicas; otherwise limits are per-process (N processes allow N× the ceiling)
PRICING_STRICT	No	<code>1</code> rejects client pricing drift with HTTP 422. <code>ecosystem.config.cjs</code> pins it to <code>1</code> in production
DB_POOL_MAX / DB_POOL_MIN	No	Defaults 50 / 4 per process; auto-derived per worker in cluster mode
AI_FEATURES_ENABLED + OPENAI_API_KEY	No	Master gate for AI slip ingestion; fail-closed when unset. <code>ANTHROPIC_API_KEY / GEMINI_API_KEY</code> optional fallbacks
IDENTITY_SSO_ENABLED + IDENTITY_*	Recommended	Enterprise SSO/MFA (OIDC) with break-glass accounts. See <code>docs/runbooks/sso-mfa-break-glass.md</code>
ALLOW_DEMO_AUTH	Never in production	Dev/test only — enables the demo password and header identity. The server refuses to boot in production if set

Boot-time secret gate: with `NODE_ENV=production` the server validates secrets before serving and exits with a clear error if `AUTH_JWT_SECRET` is missing, a placeholder, or shorter than 32 characters. It also logs `[posture]` warnings at startup for risky-but-valid settings (no durable upload storage, no `REDIS_URL` while multi-instance, SSO/MFA gaps) — review and clear these before go-live.

5.2 Generating Secure Secrets

```
# Auth token secret (also fine for SESSION_SECRET)
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
# or
openssl rand -hex 64

# Database password
openssl rand -base64 32
```

6. Application Deployment

6.1 GitHub Access — Read This Before Cloning

GitHub no longer accepts account passwords for Git operations. Attempting to clone over HTTPS and typing your GitHub password produces: `remote: Invalid username or token. Password authentication is not supported for Git operations.` Also note the repository has **moved**: the old `iwadaya/universe` and `iwadaya/claude_universe` URLs are dead — the current private repository is `Darchville-Analytics/modelling_tool`.

Option 1 — Fine-grained Personal Access Token (recommended for HTTPS):

1. On GitHub: *Settings* → *Developer settings* → *Personal access tokens* → *Fine-grained tokens* → *Generate new token*.
2. Resource owner: **Darchville-Analytics**; Repository access: *Only select repositories* → `modelling_tool`; Permissions: *Contents: Read-only*. Set a 90-day expiry (rotate per §9.2).
3. On the server, store the credential once for the deploy user, then clone. When prompted, username is your GitHub username and **the password is the token**, not your GitHub password:

```
sudo -u universe -H git config --global credential.helper store
sudo -u universe -H git clone \
  https://github.com/Darchville-Analytics/modelling_tool.git /opt/universe
# Username: <your-github-username>
# Password: <paste the fine-grained PAT>
```

The token is saved (mode 600) in `~universe/.git-credentials`, so subsequent `git pull` during deploys will not prompt.

Option 2 — SSH deploy key (no token to rotate on the server):

```
sudo -u universe -H ssh-keygen -t ed25519 -C "universe-deploy" -N "" \
  -f /home/universe/.ssh/id_ed25519
sudo -u universe -H cat /home/universe/.ssh/id_ed25519.pub
# Add this public key on GitHub: repo -> Settings -> Deploy keys ->
# "Add deploy key" (leave "Allow write access" unticked)
sudo -u universe -H git clone \
  git@github.com:Darchville-Analytics/modelling_tool.git /opt/universe
```

Migrating an existing checkout that still points at the old URL:

```
cd /opt/universe
git remote set-url origin https://github.com/Darchville-Analytics/modelling_tool.git
git fetch origin && git pull origin main
```

6.2 Repository Setup

Create a dedicated application user and deploy into `/opt/universe`. Do not run the application as root.

```
# Create dedicated system user
sudo useradd -m -s /bin/bash universe

# Clone (after configuring access per 6.1) and take ownership
sudo -u universe -H git clone \
  https://github.com/Darchville-Analytics/modelling_tool.git /opt/universe
cd /opt/universe

# Install dependencies for root, client, and server; build the frontend
npm run install:all
npm run build
```

6.3 Deployment Script

The repository ships `deploy.sh` at the repo root — use it rather than writing your own. It pulls `main`, runs `npm ci` for `root/client/server` (reproducible installs from the lockfiles), rebuilds the frontend, applies migrations, and reloads PM2:

```
chmod +x /opt/universe/deploy.sh
/opt/universe/deploy.sh
```

6.4 Database Migrations

Migrations live in `server/src/db/migrations/`, run in single transactions with savepoints, are idempotent, and are guarded by a Postgres advisory lock (cluster workers cannot double-apply). They do **not** run on boot in production — run them as an explicit pre-deploy step so a bad migration fails the deploy instead of taking the running version down:

```
npm run migrate:status --prefix server # show applied vs. pending
npm run migrate:up --prefix server # apply all pending
```

6.5 PM2 Configuration

Use the bundled `ecosystem.config.cjs` — do not start with ad-hoc flags. It configures cluster mode (workers = CPU count, capped at 4; override with `PM2_INSTANCES`), a derived per-worker `DB_POOL_MAX`, graceful reload timing, memory-limit restarts, merged logs, and `PRICING_STRICT=1`:

```
cd /opt/universe
pm2 start ecosystem.config.cjs
pm2 save
pm2 startup # follow the printed command to enable on-boot start
```

Cluster mode requires `REDIS_URL` : rate limits are per-process without it, so 4 workers would allow 4× the intended ceiling. The production posture check logs a warning if this is missed.

6.6 Alternative Runtimes

The repo also supports **Docker Compose** (bundled `Dockerfile` + `docker-compose.yml`; review before production use — the compose file carries development defaults) and **systemd** (single-process, hardened unit file example). Full recipes are in `DEPLOYMENT.md` §6.2–6.3. Pick one runtime; do not mix them.

7. Nginx Reverse Proxy and SSL

Nginx terminates HTTPS and forwards **all** traffic to the Node.js app on port 4000. Unlike v1.0, Nginx does **not** serve `client/dist` directly — Express serves the SPA itself, so a single `location /` proxy block suffices.

7.1 Nginx Site Configuration

Create `/etc/nginx/sites-available/universe`:


```

server {
    listen 80;
    server_name universe.internal.company.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name universe.internal.company.com;

    ssl_certificate      /etc/ssl/universe/cert.pem;
    ssl_certificate_key  /etc/ssl/universe/key.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    server_tokens off;
    add_header Strict-Transport-Security "max-age=31536000" always;

    client_max_body_size 50M;    # matches the app's 50 MB upload limit

    location / {
        proxy_pass http://127.0.0.1:4000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
        proxy_read_timeout 60s;
    }
}

```

```

sudo ln -s /etc/nginx/sites-available/universe /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx

```

7.2 SSL Certificate Options

Option	Guidance
Internal CA	Preferred for corporate environments. Request a certificate for the server hostname, install <code>cert.pem</code> and <code>key.pem</code>
Wildcard certificate	Recommended where <code>*.yourcompany.com</code> is already managed by IT/security
Let's Encrypt	Only if the server has public DNS + internet access: <code>sudo certbot --nginx -d universe.internal.company.com</code>
Self-signed	Development only

8. Network and Firewall Configuration

8.1 Required Ports

Port	Direction	Requirement
22 / SSH	Inbound	Admin access only; restrict to IT team IP ranges
80 / HTTP	Inbound	Redirect to HTTPS only
443 / HTTPS	Inbound	Main application access for users
4000 / Node.js	Internal only	Block external access; Nginx proxies to this port (was 3000 in v1.0)
5432 / PostgreSQL	Internal only	Block all external access; database on localhost or private subnet only
6379 / Redis	Internal only	Only if clustering; never expose externally

8.2 UFW Firewall Baseline

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.1.0/24 to any port 22    # restrict SSH to admin range
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable && sudo ufw status verbose
```

8.3 DNS and Outbound Access

- A record for `universe.internal.company.com` (or `universe.internal` for LAN-only) pointing at the server IP.
- If AI slip ingestion is enabled, the server must reach `api.openai.com` (and optionally `api.anthropic.com`, `generativelanguage.googleapis.com`).
- The server must reach `github.com` over HTTPS (or SSH port 22) for deploys.

9. Security Requirements

9.1 Access Control

Area	Requirement
SSH authentication	Key authentication only; <code>PasswordAuthentication no</code> and <code>PermitRootLogin no</code> in <code>/etc/ssh/sshd_config</code>
Application users	Real credential auth: bcrypt-hashed passwords, signed bearer tokens, role and authority re-read from the DB on every request. Roles (Actuary, Underwriter, Chief roles, Admin) managed in-app
Password policy	Enforced minimum 12 characters; seeded temp password rejected on change; common weak values rejected
SSO / MFA	Recommended for production: OIDC SSO with enforced MFA and audited break-glass accounts (<code>docs/runbooks/sso-mfa-break-glass.md</code>)
sudo access	Named administrators only; never run the app as root — use the <code>universe</code> user
File permissions	<code>chmod 600 .env</code> , <code>chmod 755 deploy.sh</code> , <code>chown -R universe:universe /opt/universe</code>

9.2 Secrets Management

Secret / asset	Requirement
<code>.env</code> file	Server only, mode 600, never committed
GitHub access	Fine-grained PAT scoped to <code>modelling_tool</code> with <i>Contents: Read-only</i> , 90-day expiry, rotate on schedule — or a read-only SSH deploy key. Never use an account password (unsupported) or a classic full-scope token
Database password	≥32 characters, stored only in <code>.env</code> (URL-encoded if needed)
<code>AUTH_JWT_SECRET</code>	≥32 chars (48 random bytes recommended); production refuses to boot without it
Rotation	Rotate all secrets every 90 days, or immediately on suspected compromise

9.3 Security Hardening Checklist

- Disable root SSH login and SSH password authentication.
- Enable automatic security updates (`unattended-upgrades`); install `fail2ban` .
- Run the app as the non-root `universe` user; keep Node/Postgres/Nginx on supported versions.
- PostgreSQL: `listen_addresses = 'localhost'` (or private subnet only); enable TLS for remote DB connections.
- Nginx: `server_tokens off` , HSTS header enabled (the app itself already ships an **enforcing nonce-based Content-Security-Policy** via Helmet — do not add a conflicting CSP in Nginx).
- Never set `ALLOW_DEMO_AUTH` or `ALLOW_OPEN_REGISTRATION` in production.

- Enable ClamAV upload scanning (`CLAMAV_ENABLED=true`) with a reachable clamd.
- Rotate or deactivate seeded accounts after first login (§11.5).

10. Monitoring and Alerting

10.1 Application Health Endpoints

Endpoint	Purpose
<code>GET /api/health</code>	Shallow health, no DB round-trip — use for load-balancer probes
<code>GET /api/health/deep</code>	Forces a Postgres round-trip, returns pool stats. <code>db.ok</code> must be <code>true</code> ; watch the <code>X-Pool-Waiting</code> header — sustained values > 0 mean <code>DB_POOL_MAX</code> needs raising

```
curl -s http://localhost:4000/api/health | jq
curl -s http://localhost:4000/api/health/deep | jq
```

10.2 PM2 and System Checks

```
pm2 monit                # real-time monitoring
pm2 status               # process status
pm2 logs universe --lines 100  # live logs (JSON to stdout)
pm2 logs universe --err --lines 50 # error logs only
free -h && uptime         # CPU / memory
df -h                   # disk
ss -tlnp | grep -E "80|443|4000|5432" # listening ports
```

10.3 Observability and Alerting

Area	Recommendation
Tracing	Set <code>OTEL_ENABLED=1</code> and <code>OTEL_EXPORTER_OTLP_ENDPOINT</code> for OpenTelemetry traces (docs/observability.md)
Metrics	Prometheus <code>/metrics</code> on <code>PROM_EXPORTER_PORT</code> (default 9464) when OTel is enabled
Uptime	Probe <code>/api/health</code> every 60 s (UptimeRobot / Grafana)
Log aggregation	Structured JSON logs to stdout; ship via PM2 → Loki/ELK; grep by <code>requestId</code>
Alerting	Alert on repeated restarts, disk > 85%, <code>[ratelimit]</code> degraded log lines, and <code>[posture]</code> startup warnings
Backup verification	<code>scripts/verify-restore.sh</code> weekly (cron) + documented monthly manual restore test

11. Go-Live Checklist

Complete and sign off every item before releasing the platform to production.

11.1 Infrastructure

- Server provisioned per §2; Ubuntu 22.04/24.04 LTS installed and updated.
- Firewall configured; 80/443 open; 4000, 5432, 6379 internal only.
- DNS record resolving; SSL certificate installed and HTTPS validated.

11.2 Software

- Node.js 20.18.0 installed (`node --version` matches `.nvmrc`).
- PostgreSQL 14+ installed and running; PM2 installed with `pm2 startup` configured; Nginx configured.
- Git access working via PAT or deploy key (§6.1); repository cloned from `Darchville-Analytics/modelling_tool` .
- Redis installed and `REDIS_URL` set if running cluster mode.

11.3 Configuration

- `.env` created from `.env.example`, permissions 600, no `CHANGE_ME` placeholders remain.
- `NODE_ENV=production`; `AUTH_JWT_SECRET` is a strong generated value; `CORS_ORIGIN` set to the public URL.
- Durable upload storage configured (`CLOUDINARY_URL`, or `ALLOW_LOCAL_UPLOADS=true` on a single persistent box).
- `ALLOW_DEMO_AUTH` and `ALLOW_OPEN_REGISTRATION` are NOT set.
- Startup log shows no unresolved [posture] warnings.

11.4 Database

- User and database created; `npm run migrate:up --prefix server` completes; `migrate:status` shows no pending migrations.
- Reference data seeded (expect 75 countries, 32 currencies on first boot).
- Backup cron installed (§4.3); first backup taken and `verify-restore.sh` exits 0.

11.5 Application

- `npm run install:all` and `npm run build` completed; PM2 running via `ecosystem.config.cjs`; `pm2 save` run.
- `deploy.sh` tested end-to-end.
- Seeded accounts handled: log in as the Chief Actuary account (`chongo.nkalamo`, temp password `Universe#1234`, forced change on first login), create real users via *Admin* → *Add User*, then rotate or deactivate every seeded account not needed.

11.6 Functional Testing

- App loads over HTTPS; login works and forces a password change on seeded accounts.
- `/api/health` and `/api/health/deep` return ok; treaty creation/saving works; NP and proportional pricing screens calculate; Actuarial Workbench loads.
- PM2 restarts the app after a manual kill test; `deploy.sh` runs clean.

12. Common Issues and Troubleshooting

12.1 Git Clone or Pull Fails: "Password authentication is not supported"

Symptom: `remote: Invalid username or token. Password authentication is not supported for Git operations. fatal: Authentication failed`.

Causes and fix, in order:

1. **Wrong URL:** the repository moved — use `https://github.com/Darchville-Analytics/modelling_tool.git`, not any `iwadaya/...` URL. For an existing checkout: `git remote set-url origin https://github.com/Darchville-Analytics/modelling_tool.git`.
2. **Password typed at the prompt:** GitHub removed password auth for Git in 2021. At the password prompt paste a **fine-grained personal access token** (§6.1 Option 1), or switch to an SSH deploy key (§6.1 Option 2).
3. **Token rejected:** confirm the token's resource owner is `Darchville-Analytics`, it includes `modelling_tool`, has *Contents: Read*, has not expired, and (for org repos) has been approved by an org admin.
4. **Prompted on every pull:** enable the credential store for the deploy user: `sudo -u universe -H git config --global credential.helper store`, then pull once and enter the token.

12.2 Application Returns 500 Errors

`pm2 logs universe --err --lines 30`. Common causes: missing `.env` variable, failed DB connection, or migrations not run.

12.3 Server Exits Immediately on Start (Production)

The boot-time secret gate is working as designed: set a strong `AUTH_JWT_SECRET` (§5.1). The startup log states exactly which check failed.

12.4 Cannot Connect to the Database

`sudo systemctl status postgresql`, then verify the URL with `psql "$DATABASE_URL" -c "SELECT 1"`. `ECONNREFUSED ::1:5432` means Postgres is down or `DATABASE_URL` is wrong. Check `pg_hba.conf` permits the connection; remember to URL-encode special characters in the password.

12.5 Nginx 502 Bad Gateway

Node is not listening on port 4000 (or `proxy_pass` doesn't match `PORT`):

```
pm2 status
ss -tlnp | grep 4000
pm2 restart universe # if stopped
```

12.6 Uploads Fail with 503 STORAGE_NOT_DURABLE

Production requires durable upload storage. Set `CLOUDINARY_URL`, or on a single box with a persistent volume set `ALLOW_LOCAL_UPLOADS=true` (§5.1). A 503 on upload with `CLAMAV_ENABLED=true` can also mean clamd is unreachable — check `CLAMAV_HOST` / `CLAMAV_PORT`.

12.7 Login Screen Lists No Users / Login Fails After Fresh Install

Reference data didn't seed or migrations didn't run. Tail the logs for migration errors and re-run `npm run migrate:up --prefix server`.

12.8 Migration Fails

Read the SQL error — it identifies the failing statement. Migrations roll back atomically, so the DB is never half-applied; `permission denied` usually means the `GRANT ALL` from §4.1 was missed. Re-run after fixing — `migrate:up` is idempotent.

12.9 PM2 App Keeps Restarting

`pm2 logs universe --err`. Typical causes: missing env variables (see 12.3), port conflict on 4000, or a worker exceeding the 1 GB memory ceiling (check for `max_memory_restart` events in `pm2 show universe`).

12.10 HTTPS Certificate Error

`sudo nginx -t`; if Let's Encrypt is used, `sudo certbot renew`.

Appendix A. Quick Command Reference

Task	Command
Run deployment	<code>/opt/universe/deploy.sh</code>
Reload app (zero downtime)	<code>pm2 reload ecosystem.config.cjs</code>
Restart app	<code>pm2 restart universe</code>
View status	<code>pm2 status</code>
View live logs	<code>pm2 logs universe --lines 100</code>
View error logs	<code>pm2 logs universe --err --lines 50</code>
Migration status	<code>npm run migrate:status --prefix server</code>
Apply migrations	<code>npm run migrate:up --prefix server</code>
Shallow / deep health	<code>curl -s localhost:4000/api/health jq / .../api/health/deep jq</code>
Test / reload Nginx	<code>sudo nginx -t / sudo systemctl reload nginx</code>
Check ports	<code>ss -tlnp grep -E "80 443 4000 5432"</code>
Backup now	<code>scripts/backup-db.sh</code> (with <code>DATABASE_URL</code> set)
Verify backups restorable	<code>scripts/verify-restore.sh</code>

Appendix B. Production Readiness Summary

The deployment is production-ready only when the following are all true:

- HTTPS works using an approved certificate; HSTS enabled; `server_tokens off`.
- SSH is key-only and restricted to authorised administrators.
- PostgreSQL and Redis are not externally exposed.
- The application runs under the non-root `universe` user via `ecosystem.config.cjs`, with `pm2 save` + `pm2 startup` done.
- `AUTH_JWT_SECRET` is a strong generated secret; no `[posture]` warnings remain at startup.
- Durable upload storage is configured; ClamAV scanning is enabled (or the risk is formally accepted).
- GitHub access uses a scoped fine-grained PAT or read-only deploy key — never an account password.
- Backups run automatically via `scripts/universe-backup.cron` and `verify-restore.sh` passes; restore tests are documented monthly.
- Seeded accounts have been rotated or deactivated; monitoring and alerting are active.
- The full go-live checklist (§11) has been signed off.